Name of the module: **MATRIX**
Location in GIT repository: vspa_common\vspa-lib\matrix

Overview:

This module will provide general purpose matrix linear algebra functions:

- Batch multiplication between a set of matrices and a set of vectors (“mat_bmult”)
- Batch multiplication with interpolation order 4: each matrix is used for 4 vector multiplications

This module is subject to future additions and changes but for now the intention is to offer support for RX equalization and TX spatial mapping.

Type of code:

- VCPU assembly functions
  - `mat_bmult_64xd2xd3_chfl_csp_chfl_asm`
  - `mat_bmult_64xd2xd3_chfx_csp_chfl_asm`
  - `mat_bmult_d1x2xd3_chfl_csp_chfl_asm`
  - `mat_bmult_d1x2xd3_chfx_csp_chfl_asm`
  - `mat_bmult_d1x1xd3_chfl_csp_chfl_asm`
  - `mat_bmult_d1x1xd3_chfx_csp_chfl_asm`
  - `mat_bmult_i4_256xd2xd3_chfl_csp_chfl_asm`
  - `mat_bmult_i4_256xd2xd3_chfx_csp_chfl_asm`
  - `mat_bmult_i4_d1x2xd3_chfl_csp_chfl_asm`
  - `mat_bmult_i4_d1x2xd3_chfx_csp_chfl_asm`
  - `mat_bmult_i4_d1x1xd3_chfl_csp_chfl_asm`
  - `mat_bmult_i4_d1x1xd3_chfx_csp_chfl_asm`
- VCPU C functions
  - `mat_bmult_chfl_csp_chfl_c`
  - `mat_bmult_chfx_csp_chfl_c`
  - `mat_bmult_i4_chfl_csp_chfl_c`
  - `mat_bmult_i4_chfx_csp_chfl_c`

Assembly Functions API:

```
void mat_bmult_64xd2xd3_chfl_csp_chfl_asm( cfloat16_t* vec_p,
                                           cfloat32_t* mat_p,
                                           cfloat16_t* out_p,
                                           uint32_t    dim2,
                                          uint32_t    dim3);

void mat_bmult_d1x2xd3_chfl_csp_chfl_asm( cfloat16_t* vec_p,
                                          cfloat32_t* mat_p,
                                          cfloat16_t* out_p,
                                          uint32_t    dim1,
                                         uint32_t    dim3);

void mat_bmult_d1x1xd3_chfl_csp_chfl_asm( cfloat16_t* vec_p,
```

```
                                        cfloat32_t* mat_p,
                                        cfloat16_t* out_p,
                                        uint32_t    dim1,
                                        uint32_t    dim3);

void mat_bmult_64xd2xd3_chfx_csp_chfl_asm( cfixed16_t* vec_p,
                                           cfloat32_t* mat_p,
                                           cfloat16_t* out_p,
                                           uint32_t    dim2,
                                           uint32_t    dim3);

void mat_bmult_d1x2xd3_chfx_csp_chfl_asm( cfixed16_t* vec_p,
                                          cfloat32_t* mat_p,
                                          cfloat16_t* out_p,
                                          uint32_t    dim1,
                                          uint32_t    dim3);

void mat_bmult_d1x1xd3_chfx_csp_chfl_asm( cfixed16_t* vec_p,
                                          cfloat32_t* mat_p,
                                          cfloat16_t* out_p,
                                          uint32_t    dim1,
                                          uint32_t    dim3);
```

vec_p – Pointer to input batch of complex vectors with size **dim1 x dim2** (vector aligned).
mat_p – Pointer to input batch of complex matrices with size **dim1 x dim3 x dim2** (vector aligned).
out_p – Pointer to output batch of complex vectors with size **dim1 x dim3** (vector aligned).
dim1 – Batch size (number of input/output vectors and number of matrices).
dim2 – Input vectors length (in samples).
dim3 – Output vectors length (in samples).

The limitations for each function:

Function name	D1 restriction	D2 restriction	D3 restriction
mat_bmult_64xd2xd3_chf*_csp_chfl_asm	D1 = 64	D2 >= 3	None (D3 >= 1)
mat_bmult_d1x2xd3_chf*_csp_chfl_asm	D1 multiple of 32	D2 = 2	None (D3 >= 1)
mat_bmult_d1x1xd3_chf*_csp_chfl_asm	D1 multiple of 32	D2 = 1	None (D3 >= 1)

```
void mat_bmult_i4_256xd2xd3_chfl_csp_chfl_asm( cfloat16_t* vec_p,
                                               cfloat32_t* mat_p,
                                               cfloat16_t* out_p,
                                               uint32_t    dim2,
                                               uint32_t    dim3);

void mat_bmult_i4_d1x2xd3_chfl_csp_chfl_asm( cfloat16_t* vec_p,
                                             cfloat32_t* mat_p,
                                             cfloat16_t* out_p,
                                             uint32_t    dim1,
                                             uint32_t    dim3);

void mat_bmult_i4_d1x1xd3_chfl_csp_chfl_asm( cfloat16_t* vec_p,
                                             cfloat32_t* mat_p,
                                             cfloat16_t* out_p,
```

```
                                            uint32_t    dim1,
                                            uint32_t    dim3);

void mat_bmult_i4_256xd2xd3_chfx_csp_chfl_asm( cfixed16_t* vec_p,
                                               cfloat32_t* mat_p,
                                               cfloat16_t* out_p,
                                               uint32_t    dim2,
                                               uint32_t    dim3);

void mat_bmult_i4_d1x2xd3_chfx_csp_chfl_asm( cfixed16_t* vec_p,
                                             cfloat32_t* mat_p,
                                             cfloat16_t* out_p,
                                             uint32_t    dim1,
                                             uint32_t    dim3);

void mat_bmult_i4_d1x1xd3_chfx_csp_chfl_asm( cfixed16_t* vec_p,
                                             cfloat32_t* mat_p,
                                             cfloat16_t* out_p,
                                             uint32_t    dim1,
                                             uint32_t    dim3);
```

vec_p – Pointer to input batch of complex vectors with size **dim1 x dim2** (vector aligned).
mat_p – Pointer to input batch of complex matrices with size **dim1/4 x dim3 x dim2** (vector aligned).
out_p – Pointer to output batch of complex vectors with size **dim1 x dim3** (vector aligned).
dim1 – Batch size (number of input/output vectors and number of matrices).
dim2 – Input vectors length (in samples).
dim3 – Output vectors length (in samples).

The limitations for each function:

Function name	D1 restriction	D2 restriction	D3 restriction
mat_bmult_i4_256xd2xd3_chf*_csp_chfl_asm	D1 = 256	D2 >= 3	None (D3 >= 1)
mat_bmult_i4_d1x2xd3_chf*_csp_chfl_asm	D1 multiple of 128	D2 = 2	None (D3 >= 1)
mat_bmult_i4_d1x1xd3_chf*_csp_chfl_asm	D1 multiple of 128	D2 = 1	None (D3 >= 1)

C Functions API:

```
MAT_BMULT_RETURN_T mat_bmult_chfl_csp_chfl_c( cfloat16_t* vec_p,
                                              cfloat32_t* mat_p,
                                              cfloat16_t* out_p,
                                              uint32_t    dim1,
                                              uint32_t    dim2,
                                              uint32_t    dim3);

MAT_BMULT_RETURN_T mat_bmult_chfx_csp_chfl_c( cfixed16_t* vec_p,
                                              cfloat32_t* mat_p,
                                              cfloat16_t  out_p,
                                              uint32_t    dim1,
                                              uint32_t    dim2,
                                              uint32_t    dim3);
```

The above C functions are wrapper functions over each set of 3 kernels with the following limitations:

Function name	D1 restriction	D2 restriction	D3 restriction
mat_bmult_chf*_csp_chfl_c	D1 multiple of 32 for D2=1,2 D1 = 64 for D2 >= 3	None (D2 >= 1)	None (D3 >= 1)

```
MAT_BMULT_RETURN_T mat_bmult_i4_chfl_csp_chfl_c( cfloat16_t* vec_p,
                                                 cfloat32_t* mat_p,
                                                 cfloat16_t* out_p,
                                                 uint32_t    dim1,
                                                 uint32_t    dim2,
                                                 uint32_t    dim3);

MAT_BMULT_RETURN_T mat_bmult_i4_chfx_csp_chfl_c( cfixed16_t* vec_p,
                                                 cfloat32_t* mat_p,
                                                 cfloat16_t  out_p,
                                                 uint32_t    dim1,
                                                 uint32_t    dim2,
                                                 uint32_t    dim3);
```

The above C functions are wrapper functions over each set of 3 kernels with the following limitations:

Function name	D1 restriction	D2 restriction	D3 restriction
mat_bmult_i4_chf*_csp_chfl_c	D1 multiple of 128 for D2=1,2 D1 = 256 for D2 >= 3	None (D2 >= 1)	None (D3 >= 1)

Implementation plan:

Throughout this document the following conventions will apply to data structures:

- A data structure of dimension N1 x N2 x N3 has the N1 dimension as the innermost and N3 as the outermost. The dimension order is the memory storage order meaning that samples are first stored along N1 dimension for N2 = 0 and N3 = 0 followed by samples along the N1 dimension for N2 = 1 and N3 = 0 and so on. The storage is assumed contiguous without gaps between subsequent dimensions.
- When using the "[]" for indexing a given dimension, the [:] will denote the entire span of that dimension.
- The notation convention of a data structure of size N1 x N2 x N3 corresponds to a Matlab convention. Such a structure will be defined in C code with inversed order of dimensions: buff[N3][N2][N1].

The input batch of vectors ("vec_p") is a dim1 x dim2 data structure:

- dim1 is the 1st dimension and dim2 is the 2nd dimension (storage order)
- dim1 is the batch size (number of input vectors) and dim2 is the length of an input vector
- a visual representation of this data structure is:

VEC[:][0]
VEC[:][1]
.....
VEC[:][D2 - 1]

where VEC[:][0] is the 1st sample from all the input vectors, VEC[:][1] is the 2nd sample from all the input vectors and so on. Memory address increases from left to right and from top to bottom. Moreover, VEC[:][n+1] is assumed stored right after VEC[:][n] without gaps.

The input batch of matrices ("mat_p") is a dim1 x dim3 x dim2 data structure:

- dim1 is the 1st dimension, dim3 is the 2nd and dim2 is the 3rd dimension (storage order)
- dim1 is the batch size (number of input matrices), dim2 is the length of an input vector and dim3 is the length of an output vector
- a visual representation of this data structure is:

MAT[:][0][0]
MAT[:][1][0]
…….
MAT[:][D3 - 1][0]
MAT[:][0][1]
MAT[:][1][1]
…….
MAT[:][D3 - 1][1]
…….
MAT[:][0][D2 - 1]
MAT[:][1][D2 - 1]
…….
MAT[:][D3 - 1][D2 - 1]

where MAT[:][0][0] is the [0][0] sample from all the input matrices, MAT[:][1][0] is the [1][0] sample from all the input matrices and so on. Memory address increases from left to right and from top to bottom. Moreover, each entry is assumed stored right after previous without gaps.

The output data structure ("out_p") is a dim1 x dim3 data structure:

- dim1 is the 1st dimension and dim3 is the 2nd dimension (storage order)
- dim1 is the batch size (number of output vectors) and dim3 is the length of an output vector
- a visual representation of this data structure is:

OUT[:][0]
OUT[:][1]
…….
OUT[:][D3 - 1]

where OUT[:][0] is the 1st sample from all the output vectors, OUT[:][1] is the 2nd sample from all the output vectors and so on. Memory address increases from left to right and from top to bottom. Moreover, OUT[:][n+1] is assumed stored right after OUT[:][n] without gaps.

The input to output relation describing the batch multiplication between a vector and a matrix is:

$$OUT[k][:]^T = MAT[k][:][:] * VEC[k][:]^T \quad k = 1 \dots D1$$

where $(\cdot)^T$ is the transpose operator used for mathematical consistency.

Another practical interpretation is that each output storage row (span along D1 dimension) is a linear combination of each input storage row (span along D1 dimension), namely:

OUT[:][0] = MAT[:][0][0] .* VEC[:][0] + MAT[:][0][1] .* VEC[:][1] + … + MAT[:][0][D2-1] .* VEC[:][D2-1]
OUT[:][1] = MAT[:][1][0] .* VEC[:][0] + MAT[:][1][1] .* VEC[:][1] + … + MAT[:][1][D2-1] .* VEC[:][D2-1]
……

where MAT[:][m][n] is the vector of coefficients multiplied with n^{th} input storage row VEC[:][n] which contributes to the m^{th} output storage row OUT[:][m].

For matrix interpolation order 4 each matrix is used for 4 multiplications, namely:

$$OUT[k][:]^T = MAT[\lfloor k/4 \rfloor][:][:] * VEC[k][:]^T \quad k = 1 \dots D1$$

where $\lfloor k/4 \rfloor$ denotes the floored division of k by 4.

Usecases:

1. **11ac RX MIMO EQUALIZER (example for 2 RX & 2 STS but not limited)**

Input RX antennas (frequency domain) -> "vec_p"

RX0[:]
RX1[:]

- the size is 64 x N_RX (with N_RX <= 8)
- dimension order is subcarrier 1st and RX 2nd:
  - "dim1" = 64 (subcarriers)
  - "dim2" = N_RX >= 1 (number of RX)
- each RX antenna contains effectively up to 64 subcarriers (e.g. 59, 62, 51) but is stored as a 64 sample buffer such that each RX buffer is **DMEM aligned**
- the precision is **16-bit Complex Half-Fixed**

W – equalization matrix -> "mat_p"

W[:][STS0][RX0]
W[:][STS1][RX0]
W[:][STS0][RX1]
W[:][STS1][RX1]

- the size is 64 x N_STS x N_RX (with N_STS <= N_RX and both N_STS, N_RX <= 8)
- dimension order is subcarrier 1st, space-time stream 2nd and RX 3rd:
  - "dim1" = 64 (subcarriers)
  - "dim3" = N_STS >= 1 (number of STS)
  - "dim2" = N_RX >= 1 (number of RX)
- the precision is **32-bit Complex Single Precision**
- the entire structure is **DMEM aligned** and since the 1st dimension is 64 each 1st dimension entry is DMEM aligned

Output space time streams (frequency domain) -> "out_p"

STS0[:]
STS1[:]

- the size is 64 x N_STS (with N_STS <= 8)

- dimension order is subcarrier 1st and space-time stream 2nd:
  - "dim1" = 64 (subcarriers)
  - "dim3" = N_STS
- the precision is **16-bit Complex Half-Float**
- each STS is **DMEM aligned**

In/out relation:

$$STS0[k] = W[k][0][0] * RX0[k] + W[k][0][1] * RX1[k]$$
$$STS1[k] = W[k][1][0] * RX0[k] + W[k][1][1] * RX1[k]$$

2. 11ac TX SPATIAL MAPPING (example for 2 STS & 2 TX but not limited)

Input STS (space-time streams, post STBC & post CSD) -> "vec_p"

STS0[:]
STS1[:]

- the size is 64 x N_STS (with N_STS <= 8)
- dimension order is subcarrier 1st and STS 2nd:
  - "dim1" = 64 (subcarriers)
  - "dim2" = N_STS >= 1 (number of STS)
- each STS is **DMEM aligned**
- the precision is **16-bit Complex Half-Float**

Q– spatial mapping matrix (frequency selective) -> "mat_p"

Q[:][TX0][STS0]
Q[:][TX1][STS0]
Q[:][TX0][STS1]
Q[:][TX1][STS1]

- the size is 64 x N_TX x N_STS (with N_STS <= N_TX and both N_STS, N_TX <= 8)
- dimension order is subcarrier 1st, TX 2nd and STS 3rd:
  - "dim1" = 64 (subcarriers)
  - "dim3" = N_TX >= 1 (number of TX)
  - "dim2" = N_STS >= 1 (number of STS)
- the precision is **32-bit Complex Single Precision**
- the entire structure is **DMEM aligned** and since the 1st dimension is 64 each 1st dimension entry is DMEM aligned

Output TX (frequency domain, prior to IFFT) -> "out_p"

TX0[:]
TX1[:]

- the size is 64 x N_TX (with N_TX <= 8)
- dimension order is subcarrier 1st and TX 2nd:
  - "dim1" = 64 (subcarriers)
  - "dim3" = N_TX

- the precision is **16-bit Complex Half-Float**
- each TX is **DMEM aligned**

In/out relation:

$$TX0[k] = Q[k][0][0] * STS0[k] + Q[k][0][1] * STS1[k]$$
$$TX1[k] = Q[k][1][0] * STS0[k] + Q[k][1][1] * STS1[k]$$

3. 11ax RX MIMO Equalizer / TX SPATIAL MAPPING

For 11ax the difference from 11ac is:

- each input RX / STS has 256 subcarriers
- each output STS / TX has 256 subcarriers
- the W/Q matrix data has 64 subcarriers and is interpolated by 4 prior to multiplication (each W/Q subcarrier coefficient is used for 4 consecutive input subcarrier)
- data allocation, ordering and alignment is preserved

DMEM requirements:

Without interpolation: *mat_bmult_**

Variable	Size (half-words)	Minimum alignment (half-words)	Allocated by
vec_p	dim1 x dim2 x 2	64	Caller
map_p	dim1 x dim2 x dim3 x 4	64	Caller
out_p	dim1 x dim3 x 2	64	Caller

With interpolation: *mat_bmult_i4_**

Variable	Size (half-words)	Minimum alignment (half-words)	Allocated by
vec_p	dim1 x dim2 x 2	64	Caller
map_p	dim1 x dim2 x dim3	64	Caller
out_p	dim1 x dim3 x 2	64	Caller

Test plan:

- Have a look in **Matrix_Testplan.xlsx** for test plan details
- Have a look in **Matrix_Report.csv** for cycle count and test status report